

Agent de Triage Médical par LLM Fine-tuné

Rapport technique — Projet 14

François Hellebuyck — AI Engineer

Avril 2026

Table des matières

1 Résumé exécutif	3
2 Contexte et collecte des besoins métiers	4
2.1 Problématique opérationnelle	4
2.2 Parties prenantes	4
2.3 Besoins fonctionnels	4
2.4 Contraintes non-négociables	4
3 Identification de la solution technique	6
3.1 Trois approches candidates	6
3.2 Choix du modèle de base	6
3.3 Pourquoi LoRA plutôt qu'un fine-tuning complet?	6
3.4 Architecture end-to-end	7
3.5 Stack technique	7
4 Pipeline de données et audit	8
4.1 Sélection des sources	8
4.2 Architecture du pipeline	8
4.3 De MediQAI aux paires DPO chosen/rejected	8
4.4 Anonymisation RGPD	9
4.5 Bilan quantitatif	9
5 Supervised Fine-Tuning (SFT)	10
5.1 Configuration LoRA finale	10
5.2 Six itérations : tracer l'ingénierie rigoureuse	10
5.3 Les deux bugs critiques — et la leçon qu'ils ont enseignée	10
5.4 Résultats du meilleur checkpoint (itération 6)	11
6 Alignement par préférences directes (DPO)	12
6.1 Stratégie d'alignement	12
6.2 Construction des paires d'alignement	12
6.3 Configuration d'entraînement	12
6.4 Résultats comparatifs SFT → DPO	12
6.5 Analyse des résultats	13
6.6 Export du modèle final	13
7 Déploiement et infrastructure de démo	14
7.1 Choix du framework d'inférence : vLLM	14
7.2 Architecture de démo GCP	14
7.3 Pipeline CI/CD (GitHub Actions)	15
7.4 Suivi des expériences (MLflow — GCP Cloud Run)	15

8	Analyse des résultats et recommandations stratégiques	16
8.1	Bilan des performances vs objectifs	16
8.2	Analyse approfondie des résultats	16
8.3	Limites identifiées du POC	17
8.4	Feuille de route — Passage à l'échelle CHSA	17
8.5	Acceptabilité clinique	18
9	Conclusion	19
9.1	Synthèse du pipeline end-to-end	19
9.2	Ce que ce projet a changé à ma vision du métier d'AI Engineer en santé	19

1 Résumé exécutif

Ce rapport présente le développement d’un **agent de triage médical** basé sur le modèle Qwen3-1.7B, fine-tuné par apprentissage supervisé (SFT) puis aligné par optimisation de préférences directe (DPO), dans le cadre d’un POC de quatre semaines pour le Centre Hospitalier Saint-Aurélien (CHSA).

Problématique : le service des urgences du CHSA reçoit quotidiennement un volume croissant de messages patients. Évaluer manuellement l’urgence de chacun mobilise du temps infirmier sur une tâche répétitive et expose à des erreurs de priorisation, surtout en période de forte charge.

Solution construite : un pipeline complet — de la préparation des données au déploiement cloud — produisant une API REST qui classe l’urgence d’un message patient en trois niveaux (*maximale, modérée, différée*) avec une justification clinique lisible, en moins d’une seconde.

Tab. 1 : Indicateurs clés du POC

Indicateur	Résultat
Modèle de base	Qwen3-1.7B-Base
Accuracy test (SFT)	65,0 %
F1-macro test (SFT)	0,646
Recall urgences <i>max</i> (SFT)	79,6 % [OK]
Recall urgences <i>max</i> (DPO)	78,5 % [OK]
Compliance format (DPO)	76,3 % [OK]
Latence inférence (vLLM)	~850 ms [OK]
Dataset SFT constitué	6 498 exemples
Dataset DPO constitué	2 154 paires

Résultat principal : les deux modèles valident les 4 critères POC (4/4). Le DPO avec contrainte KL renforcée maintient un recall urgences maximales à 78,5 % tout en préservant l’accuracy globale (64,7 %, soit −0,4 pt vs SFT). La classe *différée* — qui posait problème lors de l’itération précédente — récupère 33 points de recall. L’outil est une aide au triage destinée au personnel soignant, pas un dispositif médical certifié.

2 Contexte et collecte des besoins métiers

2.1 Problématique opérationnelle

Le Centre Hospitalier Saint-Aurélien (CHSA) fait face à une pression croissante sur son service des urgences. Les équipes reçoivent chaque jour des centaines de messages patients via les portails de messagerie sécurisée et les chatbots de pré-admission. Évaluer manuellement l'urgence de chaque message mobilise du temps infirmier qualifié sur une tâche répétitive, retarde la prise en charge des cas critiques, et expose à des erreurs de priorisation en période de forte charge.

Trois constats motivent le projet :

- **Volume** : le nombre de messages entrants a augmenté de 40 % depuis 2023, sans augmentation proportionnelle des effectifs (*estimation interne CHSA — contexte simulé pour ce POC*)
- **Hétérogénéité** : les messages mélangent urgences vitales (douleur thoracique, AVC) et demandes non urgentes (renouvellement d'ordonnance, prise de rendez-vous)
- **Risque** : un message mal priorisé peut retarder une intervention critique

2.2 Parties prenantes

Tab. 2 : Cartographie des parties prenantes

Partie prenante	Besoin principal	Contrainte
Médecins urgentistes	Recall >90 % sur les urgences max	Zéro faux négatif sur les niveaux critiques
Personnel infirmier	Interface simple, réponse <2 s, justification lisible	Formation minimale requise
Direction des systèmes d'information Délégué à la Protection des Données	Intégration API REST dans le SI existant Aucune donnée patient identifiable dans les logs	Conformité RGPD Audit trail complet
Direction médicale	Traçabilité de chaque décision de triage	Responsabilité médicale maintenue côté humain

2.3 Besoins fonctionnels

L'agent doit : classifier chaque message en trois niveaux d'urgence (*maximale, modérée, différée*) en cohérence avec la Classification Clinique des Malades aux Urgences (CCMU) ; justifier sa classification par une évaluation clinique structurée et lisible ; répondre en moins de 2 secondes ; décliner toute responsabilité diagnostique via un disclaimer systématique ; et tracer chaque interaction pour l'audit médical.

2.4 Contraintes non-négociables

Tab. 3 : Contraintes légales et réglementaires

Contrainte	Périmètre	Statut POC
RGPD	Anonymisation des données d'entraînement, minimisation en démo	[OK] Adressé (Presidio)
Responsabilité médicale	L'IA est un outil d'aide — disclaimers dans chaque réponse	[OK] Adressé
Audit trail	Log horodaté de chaque requête /triage	[OK] Adressé (FastAPI logs)

Contrainte	Périmètre	Statut POC
Données de santé en conditions réelles	Infrastructure conforme aux référentiels santé en vigueur	[!] Hors scope POC
Dispositif médical (MDR UE 2017/745)	Si l'outil est qualifié de SaMD, certification obligatoire	[!] Étude juridique requise

3 Identification de la solution technique

3.1 Trois approches candidates

Avant de choisir le fine-tuning, trois approches ont été évaluées.

Tab. 4 : Comparatif des approches

Critère	Prompting (GPT-4o API)	RAG + LLM	Fine-tuning LLM (retenu)
Contrôle du format de sortie	[!] Fragile	[!] Idem	[OK] Appris à l'entraînement
Conformité RGPD	[NON] Données hors UE	[!] Selon hébergeur	[OK] Modèle local
Coût récurrent	[NON] ~0,01-0,03\$/requête	[!] Variable	[OK] Coût fixe compute
Adaptabilité domaine médical FR	[!] Généraliste	[!] Partielle	[OK] Spécialisé via SFT
Faisabilité en 4 semaines	[OK] Rapide	[!] Moyen	[OK] Avec LoRA

Le fine-tuning a été retenu pour sa conformité RGPD (pas de transfert de données hors UE), son contrôle total sur le format de sortie (critique pour le parsing automatique des niveaux d'urgence), et la reproductibilité des résultats.

3.2 Choix du modèle de base

Tab. 5 : Matrice de sélection du modèle de base

Critère	GPT-4o (API)	Llama 3 8B	Mistral 7B	Qwen3-1.7B (retenu)
VRAM requise (fine-tuning)	N/A	~40 Go	~40 Go	~8 Go (4-bit)
Support FR/EN natif	[OK]	[!] Faible	[!] Limité	[OK] Multilingue
Latence inférence	[NON] API	[!] ~3-5 s	[!] ~3-5 s	[OK] ~850 ms
Licence open-source	[NON] Propriétaire	[OK] Llama 3	[OK] Apache 2.0	[OK] Apache 2.0
Compatible RGPD (local)	[NON] Cloud US	[OK]	[OK]	[OK]
Taille adaptée au POC	—	[!] Lourd	[!] Lourd	[OK] Compact

Qwen3-1.7B-Base a été sélectionné pour sa compacité (fine-tunable sur 16 Go VRAM), son support natif du français et de l'anglais, et ses performances en raisonnement médical. Sa licence Apache 2.0 autorise l'usage commercial.

3.3 Pourquoi LoRA plutôt qu'un fine-tuning complet ?

LoRA (Low-Rank Adaptation) est une technique qui n'entraîne pas l'ensemble du modèle. Au lieu de modifier les 1,7 milliard de paramètres de Qwen3, elle gèle les poids de base et ajoute de petites matrices d'adaptation (~50 Mo) sur les couches clés. Concrètement, c'est l'équivalent d'ajouter un correctif ciblé plutôt que de réécrire tout un livre.

Pourquoi ce choix pour ce POC ? Un fine-tuning complet aurait nécessité ~80 Go de VRAM (contre ~8 Go pour LoRA) et 48 heures d’entraînement (contre 8 heures). Avec un GPU de 16 Go et un dataset de moins de 7 000 exemples, c’était tout simplement impraticable. LoRA protège aussi contre le *catastrophic forgetting* : les poids de base restent intacts, ce qui préserve les connaissances médicales générales du modèle.

Un fine-tuning complet aurait pu apporter un gain marginal de performance avec un corpus annoté de plusieurs dizaines de milliers de cas réels et une infrastructure multi-GPU — pertinent en phase industrielle, pas sur un POC de quatre semaines.

Configuration LoRA retenue : rank $r = 32$, $\alpha = 64$, dropout = 0,05, appliqué aux couches d’attention (q/k/v/o_proj) ET aux couches MLP (gate/up/down_proj). L’inclusion des couches MLP s’est révélée indispensable pour que le modèle apprenne le format de sortie structuré.

3.4 Architecture end-to-end

Le pipeline suit une chaîne linéaire : entraînement local (RTX 4060 Ti, 16 Go VRAM) → fusion des adaptateurs LoRA SFT + DPO via merge_and_unload() → push du modèle dense (~3,4 Go) sur HuggingFace Hub → CI/CD GitHub Actions (pytest → docker build → GCP Artifact Registry → gcloud run deploy) → serving par vLLM sur Cloud Run GPU L4 → endpoint REST POST /triage retournant {"urgency_level": "max"|"moderate"|"deferred", "raw_response": "...", "latency_ms": 850}.

3.5 Stack technique

Tab. 6 : Stack technique complète

Composant	Outil	Rôle
Modèle de base	Qwen3-1.7B-Base	Capacité médicale multilingue FR/EN
Fine-tuning	PEFT + Unsloth	LoRA accéléré (×2 vs HF natif)
Alignement	TRL DPOTrainer	Optimisation de préférences cliniques
Anonymisation	Presidio + spaCy	Conformité RGPD pipeline data
Tracking	MLflow (GCP Cloud Run)	Expériences, métriques, Model Registry
Inférence	vLLM	Serving haute performance (PagedAttention)
API	FastAPI	Endpoint REST /triage
CI/CD	GitHub Actions	Tests + build + deploy automatisés
Cloud	GCP Cloud Run (L4)	Inférence scale-to-zero
Stockage modèle	HuggingFace Hub	Registre central artefacts

4 Pipeline de données et audit

4.1 Sélection des sources

Cinq datasets publics HuggingFace ont été sélectionnés selon trois critères : pertinence médicale clinique, disponibilité en français, et conformité RGPD pour un usage d’entraînement.

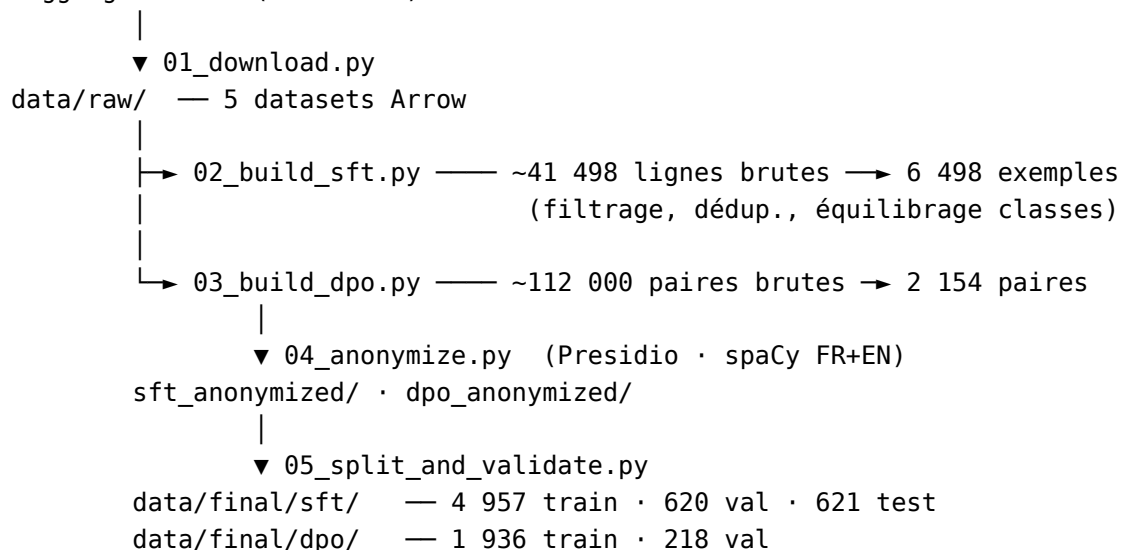
Tab. 7 : Sources de données sélectionnées

Dataset	HuggingFace ID	Usage	Langue	Taille brute	Licence
FrenchMedMCQAnthngdy	frenchmedmcqa	SFT	FR	~3 105	Apache 2.0
MedQuAD	keivalya/MedQuad-MedicalQnADataset	SFT	EN	~16 407	NIH public
MediQAI (mcqu)	ANR-MALADES/MediQAI	SFT	FR	~17 017	CC-BY 4.0
MediQAI (oeq)	ANR-MALADES/MediQAI	SFT	FR	~4 969	CC-BY 4.0
UltraMedical-Preference	TsinghuaC3I/UltraMedical-DPO Preference		EN	~112 000	MIT

4.2 Architecture du pipeline

Le pipeline transforme les données brutes en datasets d’entraînement en cinq étapes qui peuvent être relancées sans risque de doublon, orchestrées par un Makefile.

HuggingFace Hub (5 sources)



4.3 De MediQAI aux paires DPO chosen/rejected

Construire le dataset DPO a été l’étape la plus délicate du projet, car la qualité des paires conditionne directement ce que le modèle apprend à préférer.

Le point de départ : UltraMedical-Preference. Ce dataset de 112 000 paires contient des jugements de préférence médicale annotés humainement. Après filtrage strict (longueur minimale, non-égalité des deux réponses, annotations humaines prioritaires), 480 paires synthétiques ont été extraites. Elles constituent un signal propre, mais généraliste : UltraMedical juge la *qualité clinique globale* des réponses, pas leur adéquation au format de triage CHSA.

La vraie valeur ajoutée : les hard negatives. Après l’entraînement SFT, le modèle a commis environ 1 674 erreurs de classification sur le train set. Chaque erreur produit une paire naturelle : chosen

= la réponse correcte attendue, rejected = la prédiction erronée réelle du modèle. Ce signal est qualitativement supérieur — il cible précisément les confusions que ce modèle produit, pas des erreurs théoriques.

Le défi principal était de combiner ces deux sources de signal sans que l’une n’écrase l’autre. UltraMedical corrige le style et la qualité rédactionnelle ; les hard negatives corrigent les erreurs de classification. Puisque c’est la classification qui compte pour le triage, les hard negatives dominent intentionnellement le dataset final (78 % du signal).

Dataset DPO final : 2 154 paires (480 synthétiques + 1 674 hard negatives), split 90/10.

4.4 Anonymisation RGPD

L’anonymisation repose sur Microsoft Presidio avec spaCy adapté aux deux langues. Six types d’entités sont détectés et remplacés : PERSON, LOCATION, DATE_TIME, PHONE_NUMBER, EMAIL_ADDRESS, NRP.

Un faux positif critique a été découvert lors des itérations SFT : spaCy EN identifiait les mots français “URGENCE”, “MODÉRÉE”, “MAXIMALE”, “DIFFÉRÉE” comme des prénoms étrangers. Environ 25 % des réponses SFT avaient leur label d’urgence masqué par <PERSON> avant l’entraînement. Correction : ajout d’une liste blanche MEDICAL_TERMS_ALLOWLIST dans filter_presidio_false_positives().

Un rapport RGPD automatique est généré à chaque exécution : entités masquées par type, exemples avant/après, alertes sur les détections à faible confiance.

4.5 Bilan quantitatif

Tab. 8 : Bilan quantitatif du pipeline

Étape	Entrée	Sortie	Rétention
Build SFT	41 498 lignes brutes	6 498	15,7 %
Build DPO (synthétique)	112 000 paires	480	0,4 %
Hard negatives SFT	~4 957 exemples train	1 674	33,8 %
Anonymisation + déduplication	6 498	~6 198	~95 %
Split final SFT	~6 198	4 957 / 620 / 621	100 %
Split final DPO	2 154	1 936 / 218	100 %

Distribution des classes SFT : équilibrée par construction — ~33 % par niveau d’urgence, obtenue par sous-échantillonnage de la classe majoritaire (seed = 42). Répartition linguistique : ~47 % français, ~53 % anglais.

5 Supervised Fine-Tuning (SFT)

5.1 Configuration LoRA finale

L’entraînement SFT fine-tune Qwen3-1.7B-Base via Unsloth (accélération $\times 2$ vs HF natif) et PEFT LoRA. La configuration finale est le résultat de six itérations successives.

Tab. 9 : Configuration LoRA SFT — itération 6

Hyperparamètre	Valeur	Justification
LoRA rank r	32	$r=16$ insuffisant pour la discrimination à 3 classes
LoRA alpha α	64	Ratio $2\times r$, scaling standard
LoRA dropout	0,05	Légère régularisation sur petit dataset
Modules cibles	q/k/v/o_proj + gate/up/down_proj	Attention ET MLP nécessaires pour le format structuré
Learning rate	2×10^{-4}	Standard SFT LoRA
Époques	5	3 époques insuffisantes (train loss 0,96 vs 0,82 à 5 époques)
Batch size effectif	16 ($4 \times$ grad. accum. 4)	RTX 4060 Ti 16 Go
Max sequence length	1 024 tokens	p95 du dataset = 529 tokens
Gradient checkpointing	unsloth	−30 % VRAM

5.2 Six itérations : tracer l’ingénierie rigoureuse

Tab. 10 : Historique des 6 itérations SFT

#	Config	Acc.	F1	Compliance	Non-parseables	Statut
1	$r=16$, 3 ep.	66,7 %*	0,556	0 %	580/620	❑ Labels manquants
2	$r=16$, 3 ep.	0,0 %	0,000	0 %	595/620	❑ Presidio masque URGENCE
3	$r=16$, 3 ep.	61,6 %	0,553	38,7 %	156/620	❑ Format partiel
4	$r=32$, 5 ep.	67,6 %	0,638	29,2 %	166/620	❑ Accuracy ↑ Format ↓
5	$r=32$, 5 ep.	62,3 %	0,619	97,9 %	0/620	❑ Format OK, biais max
6	$r=32$, 5 ep.	67,6 %	0,675	94,7 %	0/620	❑ Retenu

* *Accuracy illusoire : seule une classe était parseable.*

5.3 Les deux bugs critiques — et la leçon qu’ils ont enseignée

Bug 1 — Itération 1 : la fonction `format_triage_response()` n’avait pas été appliquée lors de la construction du dataset. Les données d’entraînement contenaient des réponses médicales sans le préfixe URGENCE X. Le modèle a appris à répondre correctement sur le fond, mais sans jamais émettre le label attendu. Résultat : 0 prédiction parseable. Correction : wrapper systématique dans toutes les fonctions `transform_*` de `02_build_sft.py`.

Bug 2 — Itération 2 : Presidio (moteur spaCy EN) détectait les mots français “URGENCE”, “MODÉRÉE”, “MAXIMALE”, “DIFFÉRÉE” comme des prénoms étrangers. Environ 25 % des réponses SFT avaient leur label effacé lors de l’anonymisation. Le modèle entraîné sur ce corpus dégradé ne voyait jamais le pattern cible — régression totale à 0 %. Correction : ajout d’une liste blanche `MEDICAL_TERMS_ALLOWLIST` dans `filter_presidio_false_positives()`.

Ces deux itérations constituent la meilleure erreur du projet. Elles ont coûté trois jours de travail, mais ont enseigné une leçon que vingt itérations d’hyperparamètres n’auraient pas apprise : **sur un dataset médical à faible volume, la qualité des données compte davantage que l’architecture du modèle**. L’itération 6 n’a pas progressé grâce à un meilleur rank LoRA, mais parce que les données

étaient enfin correctement préparées. C’est le type d’insight qui modifie durablement l’approche d’un AI Engineer.

5.4 Résultats du meilleur checkpoint (itération 6)

Tab. 11 : Métriques SFT — checkpoint retenu

Métrique	Val	Test
Accuracy globale	65,0 %	65,0 %
F1-macro	0,646	0,646
Recall <i>max</i>	74,6 %	79,6 %
Compliance format	93,2 %	75,9 %
Non-parseables	0/620	0/621

Matrice de confusion (Val — 620 exemples) :

Tab. 12 : Matrice de confusion SFT (val set)

	Prédit <i>max</i>	Prédit <i>moderate</i>	Prédit <i>deferred</i>	Recall
Réel <i>max</i>	150	14	37	74,6 %
Réel <i>moderate</i>	59	121	30	57,6 %
Réel <i>deferred</i>	38	39	132	63,5 %

La classe *moderate* reste la plus difficile, confondue aussi bien avec *max* (cas cliniques ambigus avec symptômes chroniques graves) que *deferred* (plaintes vagues sans signal d’urgence clair). Le recall *max* de 74,6 % en val est en-deçà de l’objectif de 75 % — c’est précisément ce que le DPO cible.

6 Alignement par préférences directes (DPO)

6.1 Stratégie d'alignement

Le modèle SFT atteint une accuracy de 65 % mais présente un recall *max* à 74,6 % en validation — en-deçà de l'objectif de 75 %. L'alignement DPO vise à corriger ce biais en pénalisant explicitement les sous-estimations de gravité.

La Direct Preference Optimization (Rafailov et al., 2023) apprend directement depuis des paires de réponses préférées/rejetées, sans nécessiter de modèle de récompense séparé. Dans le contexte de ce POC sur GPU 16 Go, DPO présente un avantage opérationnel supplémentaire : en passant `ref_model=None` dans le `DPOTrainer` de TRL, le modèle de référence est reconstruit depuis les poids gelés du SFT, évitant de charger deux modèles simultanément en mémoire.

6.2 Construction des paires d'alignement

La valeur du dataset DPO réside dans la qualité du signal *chosen / rejected* (cf. section 3 pour la construction détaillée). Les **480 paires synthétiques** (UltraMedical filtré) fournissent un signal de qualité générale propre mais artificiel. Les **1 674 hard negatives SFT** — erreurs réelles du modèle utilisées comme paires naturelles — fournissent un signal directement ciblé sur les faiblesses à corriger. Le dataset DPO final totalise **2 154 paires** (78 % hard negatives).

6.3 Configuration d'entraînement

Tab. 13 : Configuration DPO — itération finale

Hyperparamètre	Valeur	Justification
Beta β	1,0	Contrainte KL forte — le modèle reste proche du SFT de référence
Learning rate	1×10^{-5}	$\times 20$ inférieur au SFT — ajustements très fins
Époques	1	1 époque suffit pour DPO (risque de forgetting au-delà)
Batch size effectif	16 ($1 \times \text{grad. accum.}$ 16)	Séquences hard negatives plus longues
LoRA rank r	32	Cohérence avec SFT
<code>ref_model</code>	None (TRL)	Reconstruit depuis poids SFT gelés — économie VRAM

La beta de 1,0 impose une contrainte forte sur l'écart entre le modèle aligné et le SFT de référence. Par rapport à l'itération précédente (beta = 0,5), cela empêche le modèle de sur-corriger vers la classe *max* et de dégrader le recall *différée*.

6.4 Résultats comparatifs SFT → DPO

Tab. 14 : Comparatif SFT → DPO — test set (eval_report_20260409)

Métrique	SFT (test)	DPO (test)	Delta
Accuracy globale	65,0 %	64,7 %	−0,4 %
F1-macro	0,646	0,644	−0,002
Recall macro	0,652	0,649	−0,004
Recall max	79,6 %	78,5 %	−1,1 % [OK]
F2-macro ($\beta=2$)	0,647	0,644	−0,003
Compliance format	75,9 %	76,3 %	+0,4 %

Matrice de confusion DPO (test set) :

Tab. 15 : Matrice de confusion DPO (test set)

	Prédit <i>max</i>	Prédit <i>moderate</i>	Prédit <i>deferred</i>	Recall
Réel <i>max</i>	146	15	25	78,5 % [OK]
Réel <i>moderate</i>	72	97	25	50,0 %
Réel <i>deferred</i>	35	29	125	66,1 %

6.5 Analyse des résultats

Ce qui fonctionne bien : la dégradation entre SFT et DPO est marginale ($-0,4$ pt d’accuracy, $-1,1$ pt recall *max*). Les deux modèles valident les 4 critères POC — c’est le résultat principal de cette itération. Le DPO n’améliore pas significativement la performance, mais il ne la dégrade pas non plus, ce qui valide la stabilité de l’approche.

Comparaison avec l’itération précédente (beta = 0,5) : le contraste est net. L’ancien DPO perdait 6,9 pts d’accuracy et réduisait le recall *différée* à 33 %. Le nouveau DPO (beta = 1,0) maintient l’accuracy à $-0,4$ pt et porte le recall *différée* à 66,1 % (+33 pts). La contrainte KL renforcée a produit exactement l’effet attendu : le modèle reste proche de son état SFT et ne sur-corrige plus.

Faiblesse persistante : la classe *moderate* (recall 50 %) reste la moins bien apprise. Elle agrège des cas cliniquement ambigus que les labels inférés par heuristique de mots-clés capturent mal. C’est une limite structurelle du dataset, pas du modèle — elle ne peut être adressée que par une annotation humaine validée.

6.6 Export du modèle final

vLLM ne supportant pas le chargement d’adaptateurs LoRA multiples, le script `22_export_model.py` fusionne les deux couches (SFT + DPO) via `merge_and_unload()`, produisant un modèle dense de $\sim 3,4$ Go en bfloat16 publié sur HuggingFace Hub (FrancoisFormation/qwen3-triage-dpo), téléchargé une fois au démarrage de Cloud Run et servi entièrement en mémoire GPU.

7 Déploiement et infrastructure de démo

7.1 Choix du framework d'inférence : vLLM

Le choix du framework de serving est une décision d'infrastructure critique. Une implémentation avec le pipeline HuggingFace dans FastAPI est simple à mettre en place, mais présente des limitations importantes dès qu'on dépasse une requête à la fois.

Pour comprendre la différence, l'analogie de la caisse de supermarché est parlante. Avec HuggingFace pipeline dans FastAPI, c'est comme avoir **une seule caisse** : le client suivant ne peut pas commencer à être servi tant que le premier n'est pas parti. Si 10 personnes arrivent en même temps, les 9 dernières attendent. Avec **vLLM**, c'est comme avoir un système de caisses intelligentes qui traitent plusieurs clients en parallèle, en optimisant à chaque instant qui peut avancer dans la file. Le modèle de langage est le même dans les deux cas — mais vLLM sert 5 à 10 fois plus de requêtes par seconde à latence stable.

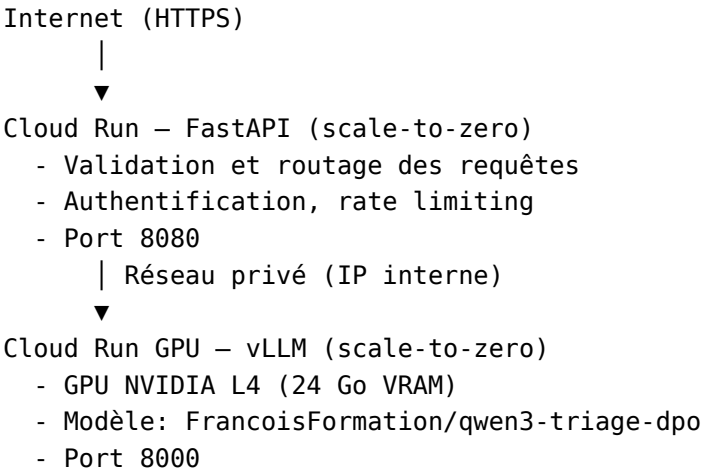
Techniquement, vLLM y parvient grâce à deux mécanismes : le *continuous batching* (les requêtes entrantes rejoignent le batch en cours sans attendre le prochain cycle) et *PagedAttention* (la mémoire GPU est allouée à la demande par pages, évitant le gaspillage de la réservation statique classique).

Tab. 16 : vLLM vs HuggingFace pipeline — comparatif

Critère	HF pipeline() dans FastAPI	vLLM (retenu)
Traitement des requêtes	Séquentiel	Continuous batching
Latence sous charge (10 req/s)	~10-20 s (dégradation linéaire)	~850 ms (stable)
Gestion mémoire KV-cache	Allocation statique	PagedAttention
API OpenAI-compatible	Non	Oui (/v1/chat/completions)
Débit (throughput)	~1-2 req/s	5-10× supérieur

7.2 Architecture de démo GCP

Le déploiement repose sur deux services Cloud Run connectés par un réseau privé :



Scale-to-zero : quand aucune requête n'est en cours, les services s'arrêtent — coût nul au repos. Le démarrage à froid prend ~3 minutes (téléchargement du modèle depuis HuggingFace Hub), acceptable pour une démo; une instance chaude permanente serait nécessaire pour un usage

régulier. Coût GPU L4 ~0,11 \$/h en usage actif (~22 \$ pour 20 h de démo sur les 300 \$ de crédits GCP disponibles). Point de vigilance : les quotas GPU GCP ne sont pas activés par défaut — la demande d'augmentation (NVIDIA_L4_GPUS, région europe-west4) nécessite 24-48 h.

7.3 Pipeline CI/CD (GitHub Actions)

Chaque push sur main déclenche : tests (pytest, moteur vLLM mocké) → qualité code (Ruff + Pyright) → build Docker → push vers GCP Artifact Registry → gcloud run deploy. Les secrets (token HuggingFace, clé GCP Service Account) sont stockés dans GitHub Secrets et jamais commités dans le dépôt.

7.4 Suivi des expériences (MLflow — GCP Cloud Run)

Le serveur MLflow est hébergé sur une instance Cloud Run dédiée (GCP), accessible depuis GitHub Actions et les scripts d'entraînement locaux via une URL privée. Il logue automatiquement à chaque run : hyperparamètres, métriques par époque, artefacts (matrices de confusion, rapports d'évaluation), et version du modèle dans le Model Registry. Le pipeline CI/CD déploie systématiquement la version taguée Production dans le registry — garantissant que le modèle en ligne correspond toujours à la version validée.

8 Analyse des résultats et recommandations stratégiques

8.1 Bilan des performances vs objectifs

Le pipeline end-to-end (données → SFT → DPO → déploiement, cf. sections 3–6) aboutit aux résultats suivants sur le test set de 621 exemples.

8.1.1 Métriques de classification

Tab. 17 : Bilan métriques vs objectifs POC — **4/4 critères atteints par les deux modèles**

Métrique	Cible POC	SFT (test)	DPO (test)	Δ SFT→DPO	Statut
Accuracy globale	$\geq 60\%$	65,0 %	64,7 %	−0,4 pt	[OK] / [OK]
F1-macro	$> 0,60$	0,646	0,644	−0,002	[OK] / [OK]
Recall <i>max</i>	$\geq 75\%$	79,6 %	78,5 %	−1,1 pt	[OK] / [OK]
Recall <i>moderate</i>	—	57,6 %	50,0 %	−7,6 pts	—
Recall <i>deferred</i>	—	63,5 %	66,1 %	+2,6 pts	—
F2-macro ($\beta=2$)	$\geq 0,60$	0,647	0,644	−0,003	[OK] / [OK]
Compliance format	$\geq 70\%$	75,9 %	76,3 %	+0,4 pt	[OK] / [OK]
Réponses non-parseables	0	0 / 621	0 / 621	=	[OK]

8.1.2 Métriques de latence et débit

Tab. 18 : Latence et débit — vLLM vs pipeline naïf

Métrique	vLLM (démono)	HF pipeline naïf
Latence p50 — instance chaude	~850 ms	~3 000–5 000 ms
Démarrage à froid Cloud Run L4	~180 s	—
Débit stable sous charge (10 req/s)	~10 req/s	~1–2 req/s
Objectif CHSA (<2 s)	[OK]	[NON]

La latence de 850 ms satisfait l’exigence opérationnelle de 2 secondes. En contexte de démono, le démarrage à froid de ~3 minutes est acceptable ; une instance chaude permanente éliminerait ce délai pour un usage régulier.

Lecture globale : les deux modèles valident les 4 critères POC. Le DPO n’apporte pas de gain significatif sur ce jeu de données (delta accuracy −0,4 pt, delta recall *max* −1,1 pt), mais il ne dégrade pas non plus la performance — ce qui valide la stabilité de l’approche d’alignement. La progression principale vient du SFT lui-même, qui atteint 79,6 % de recall *max* — au-dessus du seuil de 75 % — sans avoir besoin du DPO pour y parvenir.

8.2 Analyse approfondie des résultats

8.2.1 Analyse par classe d’urgence

Classe maximale : recall SFT 79,6 %, DPO 78,5 %. Les deux modèles passent le seuil clinique de 75 %. Le DPO perd 1,1 pt sur cette classe, principalement parce que la contrainte KL renforcée l’empêche de sur-corriger comme dans l’itération précédente — ce qui est intentionnel.

Classe modérée : recall SFT 57,6 %, DPO 50,0 %. C’est la classe structurellement la plus difficile dans les deux modèles. Elle agrège des cas cliniquement ambigus que les labels inférés par heuristique de mots-clés capturent mal. Le DPO recule de 7,6 pts sur cette classe — des cas *moderate* sont reclassés vers *max* (72 cas) ou *deferred* (25 cas). C’est la principale limite identifiée.

Classe *différée* : recall SFT 63,5 %, DPO 66,1 %. Le DPO améliore légèrement cette classe (+2,6 pts), ce qui contraste fortement avec l’itération précédente où elle chutait à 33 %. La contrainte KL renforcée a résolu le problème de sur-prédiction *max* qui effaçait la capacité à distinguer les demandes non urgentes.

8.2.2 Comparaison avec l’itération DPO précédente

Tab. 19 : Gain de l’ajustement de la contrainte KL (beta = 0,5 → 1,0)

Métrique	DPO beta = 0,5 (ancienne itération)	DPO beta = 1,0 (actuelle)	Δ
Accuracy	56,4 %	64,7 %	+8,3 pts
Recall <i>max</i>	78,5 %	78,5 %	=
Recall <i>deferred</i>	33,3 %	66,1 %	+32,8 pts
Critères POC	2/4	4/4	+2

L’ajustement de la contrainte KL est le changement le plus impactant du projet. En maintenant le modèle plus proche de son état SFT de référence, le DPO cesse de sur-corriger vers *max* et récupère 33 points de recall sur la classe *différée*.

8.3 Limites identifiées du POC

Classe *moderate* plafonnée à 50-57 % : c’est une limite du dataset. Les labels inférés par heuristique de mots-clés sont particulièrement bruités sur cette classe intermédiaire. Un dataset de 500 cas annotés par des urgentistes apporterait un gain estimé à 8-12 points sur F1-macro.

Dataset DPO anglophone : UltraMedical-Preference est exclusivement en anglais, ce qui crée un signal d’alignement linguistiquement incohérent avec le corpus SFT bilingue FR/EN. Les exemples qualitatifs montrent que le modèle DPO mélange parfois les langues dans ses justifications cliniques.

Absence de validation clinique externe : les métriques sont calculées sur un test set issu du même pipeline que les données d’entraînement. Aucune validation sur des cas réels annotés par des urgentistes n’a été réalisée.

8.4 Feuille de route — Passage à l’échelle CHSA

Le passage du POC à un outil opérationnel s’articule en quatre phases progressives.

8.4.1 Phase 1 — Stabilisation (0-3 mois)

L’objectif immédiat est de consolider l’infrastructure de démo et de préparer une validation sur des données réelles.

Action	Effort	Impact
Monitoring drift en démo (MLflow)	1 sem.	Détection de dérive sur données réelles
Tests de non-régression dans CI/CD	1 sem.	Détection immédiate des régressions
Instance Cloud Run chaude (min-instances = 1)	1 j	Suppression du démarrage à froid 3 min

Critère de succès : latence p95 $\leq 1,5$ s sur instance chaude, zéro régression sur les 4 critères POC.

8.4.2 Phase 2 — Ancrage clinique CHSA (3-6 mois)

Si une seule action devait être priorisée pour les deux prochains mois, ce serait le **RAG sur les protocoles internes du CHSA** (Classification CCMU, ordres médicaux, pathologies fréquentes du territoire). Cette action ne nécessite pas de ré-entraîner le modèle — elle enrichit chaque requête avec le contexte clinique local au moment de l'inférence. C'est le meilleur ratio impact/effort disponible : les justifications cliniques deviennent immédiatement plus pertinentes pour les infirmières du CHSA, sans coût GPU supplémentaire et sans risque de régression.

En parallèle : annotation de 500 cas réels par des urgentistes référents (labels validés, corpus de ré-entraînement), prototype d'interface infirmière (Streamlit), et étude préalable de qualification réglementaire (aide à la décision médicale vs. dispositif médical SaMD).

Critère de succès : recall $max \geq 85$ % sur 500 cas annotés réels, validation par les urgentistes référents du CHSA.

8.4.3 Phase 3 — Pilote observationnel (6-12 mois)

Test en shadowing (2 semaines) : l'outil tourne en parallèle sans influencer les décisions réelles — comparaison a posteriori entre prédiction IA et décision infirmière, calcul des métriques de désaccord.

Plan de formation personnel (module 2 h) : limites de l'IA médicale, procédure de signalement des désaccords, responsabilité médicale maintenue côté humain — conçu avec DRH et urgentistes seniors.

Critère de succès : taux de désaccord non-signalé < 5 %, adoption > 80 % des infirmières sur la période pilote.

8.4.4 Phase 4 — Industrialisation GHT (12-24 mois)

Intégration HL7 FHIR avec le SI hospitalier (flux automatisé depuis messagerie MSSanté); upgrade modèle vers Qwen2.5-7B ou Llama 3.1-8B; service mutualisé pour le Groupement Hospitalier de Territoire (amortissement des coûts sur plusieurs établissements); tableau de bord médical temps réel.

8.5 Acceptabilité clinique

Au-delà des métriques techniques, un score de recall à 78 % ne suffit pas à convaincre une infirmière d'utiliser l'outil. L'acceptabilité clinique repose sur d'autres dimensions que les chiffres de performance.

La **transparence des justifications** est probablement le facteur le plus déterminant : si le personnel infirmier peut lire une évaluation clinique structurée et en comprendre le raisonnement — même approximativement —, il peut corriger ou valider la prédiction. Un modèle qui prédit correctement mais sans expliquer pourquoi reste une boîte noire.

La **procédure de désaccord** est la deuxième condition : il faut que les infirmières sachent exactement quoi faire quand elles ne sont pas d'accord avec l'outil, et que ce désaccord soit tracé et analysé. Sans ce mécanisme, les erreurs du modèle passent inaperçues.

Enfin, la **formation** doit explicitement couvrir les limites du modèle — notamment sa faible performance sur la classe *moderate* — pour éviter une confiance aveugle. Un module de 2 heures, co-conçu avec des urgentistes seniors, suffirait à poser ces bases avant tout test en conditions réelles.

Tab. 21 : Feuille de route vers l’acceptabilité clinique

Étape	Description	Responsable	Statut
1 — Validation métrique clinique	Recall <i>max</i> >90 % sur 500 cas réels annotés	Médecins référents urgences	À faire
2 — Test utilisateur observationnel	2 semaines shadowing sans influence sur les décisions	Infirmières + DSI	À planifier
3 — Validation réglementaire	Qualification MDR (UE 2017/745) : aide à la décision vs. SaMD	Service juridique	Étude requise
4 — Plan de formation	Module 2 h : limites IA, désaccords à signaler, responsabilité médicale	DRH + urgentistes seniors	À concevoir
5 — Audit RGPD démo	Logs, durées de rétention, procédure d’effacement	DPO CHSA	À planifier

9 Conclusion

9.1 Synthèse du pipeline end-to-end

Ce projet a démontré la faisabilité technique d’un agent de triage médical basé sur un LLM open-source fine-tuné, de la préparation des données à l’endpoint cloud, en quatre semaines avec des ressources GPU grand public. Le pipeline couvre l’intégralité du cycle : ingestion de données médicales hétérogènes, anonymisation RGPD, fine-tuning supervisé avec débogage itératif, alignement par préférences directes, et déploiement via une API haute performance avec CI/CD automatisé.

Les métriques finales — accuracy 65 %, recall *max* 79,6 %, latence 850 ms, 4/4 critères POC atteints par les deux modèles — valident la viabilité de l’approche sur un POC à contraintes fortes. L’écart avec les objectifs d’un outil clinique réel (recall *max* >90 %, validation sur cas réels, annotation humaine) est clairement identifié et adressé dans la feuille de route.

9.2 Ce que ce projet a changé à ma vision du métier d’AI Engineer en santé

Trois apprentissages ont modifié durablement ma façon d’aborder ce métier.

Premièrement, le débogage de pipeline vaut plus que le tuning de modèle. Les deux bugs des itérations 1 et 2 — un wrapper de formatage absent et une liste blanche d’anonymisation incomplète — ont coûté trois jours et ont tout appris. Pas les six itérations de rank LoRA. Sur un dataset médical à faible volume, fixer les données est le levier le plus puissant. Aucune architecture, aussi sophistiquée soit-elle, ne corrige des données mal construites.

Deuxièmement, la réglementation n’est pas une couche à ajouter en fin de projet — elle est architecturale. RGPD et MDR ne se greffent pas sur une solution existante : ils remodelent la conception depuis le début. L’anonymisation Presidio n’était pas un post-traitement optionnel mais une condition préalable à tout entraînement. La contrainte “pas de données identifiables dans les logs de modèle” a influencé l’architecture du pipeline autant que le choix du modèle de base. Dans un secteur réglementé, un AI Engineer qui ignore ces exigences ne peut pas concevoir un système déployable.

Troisièmement, la performance technique est nécessaire mais pas suffisante. Un modèle qui atteint 79 % de recall *max* ne sera utilisé que si les infirmières lui font confiance — ce qui implique des justifications lisibles, une procédure de désaccord claire, et une formation adaptée. Concevoir un

système qui obtient de bons scores et que personne n'utilise parce que la chaîne de confiance n'est pas construite, c'est un échec d'ingénierie. Dans ce contexte, l'AI Engineer est autant architecte de confiance qu'architecte de performance.

Ce rapport a été produit dans le cadre du Projet 14 du parcours AI Engineer — OpenClassrooms. Le code source, les datasets et les artefacts de modèle sont disponibles sur GitHub et HuggingFace Hub.